

# Parallel and Distributed Computing Assignment Report

## Overview

This project focuses on implementing a particle simulation in a two-dimensional space, with the objective of exploring different computational approaches for solving the  $n$ -body problem. The simulation models the movement of particles under the influence of gravitational forces, using a Particle-in-Cell (PIC) approximation to reduce computational complexity.

The project consists of three main implementations:

- A serial implementation, serving as the baseline for performance comparison.
- A parallel implementation using OpenMP, optimizing computation for shared-memory architectures.
- A distributed-memory implementation using MPI, enabling scalability across multiple compute nodes.

The goal is to analyze and compare the efficiency of these implementations by measuring execution time across different problem sizes and parallel configurations. The project also involves addressing key challenges such as load balancing, synchronization, and communication overhead.

The implementation is done in C, utilizing OpenMP and MPI for parallelism. The report will include a detailed discussion on the methodologies employed, performance evaluations, and insights gained from the parallelization strategies.

## Numerical Precision Error in Serial and OpenMP Code

In the initial implementation of both the serial and OpenMP versions presented, a subtle numerical instability due to the use of AVX2 SIMD instructions was found. While the code's vectorization provided significant performance improvements, one test case that produced inconsistent collision counts, due to floating-point arithmetic behavior, was identified.

This instability originated from the manner in which AVX2 instructions restructured floating-point operations, particularly in reduction patterns. Unlike scalar operations that follow strict sequential computations, SIMD vectorization processes multiple values concurrently. This alters the operation order, resulting in different rounding patterns during finite-precision floating-point computations. Over several simulation steps, these minor discrepancies added up, are just enough to change whether some particles were counted as collided, or not, especially when particles came extremely close without quite touching.

After further analysis, it was discovered that when multiple threads concurrently add particles to the same cell, the order of particles within the cell can vary between runs. This nondeterministic ordering can, in some cases, lead to differences in the computed positions and velocities of certain particles, which may ultimately result in a different total number of collisions.

Since addressing these behaviors would incur significant performance overhead, the group has decided to not modify the code and to retain the current implementation despite the precision issues.

## MPI Implementation

In designing the MPI-based implementation, we followed the structured approach of **Foster's Design Methodology**, which consists of four main stages: *Partitioning*, *Communication*, *Agglomeration*, and *Mapping*. Below is a breakdown of how each stage was applied to our simulation.

- **Partitioning:** The simulation domain was divided into a cell grid, and each cell served as a basic unit of computation. These cells contained varying numbers of particles, which were the primary data entities.
- **Communication:** Since particles could move across cell boundaries, communication between neighboring grid sections is done at each time step, sending and receiving the center of mass and particles too.
- **Agglomeration:** Individual grid cells were grouped into larger blocks to reduce communication overhead. Initially, subdomains were grouped row-wise across MPI processes, simplifying communication patterns. Later, a checkerboard-style distribution was implemented. Although this introduced more complex communication

patterns, it significantly reduced communication overhead due to the decreased perimeter of square regions compared to long rows. This approach was especially advantageous as the number of processors increases.

- **Mapping:** The task assigned to each processors are represented by block chunks of cells agglomerated in squares.

To handle the communication between blocks several additions are necessary:

- **Multiple Buffers and Locks per Neighbor:** Each neighbor has a dedicated set of buffers and corresponding locks to avoid contention during particle exchange.
- **Routing Logic for Outgoing Particles:** Additional logic determines the appropriate buffer for each particle that leaves its cell, ensuring it is sent to the correct neighbor.
- **Thread-Safe Access with Locks:** When using OpenMP, locks are used to ensure safe concurrent access to the shared buffers associated with each neighbor..

For further improve the code, several optimizations are used:

- **Pre-Allocated Buffers:** Memory for particle buffers is allocated once at the beginning of the simulation, minimizing performance issues caused by memory reallocation during runtime.
- **Initializing Receiver/Sender:** Since the number of exchanged centers of mass remains constant at every timestep, MPI buffers are initialized once using *MPI\_Recv\_init()* and *MPI\_Send\_init()* and then used several times with *MPI\_Start()*.
- **Dynamic Scheduling for Computation:** Dynamic Scheduling for Computation: Because each cell contains a varying number of particles, dynamic scheduling is used when performing cell computations (e.g., gravitational forces, particles migration, and collision detection). The chunk size is configurable: it is set to 1 when the number of cells per thread is relatively small, otherwise 2.
- **Static Scheduling for Communication:** The communication phase — especially receiving data from other nodes — is typically limited by network latency rather than memory copying speed. So in this case, using a dynamic scheduler doesn't help to reduce the execution time, so a static block scheduler is used to reduce scheduling overhead.
- **Overlapping Communication with Computation:** Receiver requests are initiated as early as possible to hide communication latency with computation. When waiting for sender request completions, only one thread blocks on the communication while others continue with computational intensive operations. Then, using dynamic scheduling allows the maximum amount of work be finished before the sender requests finishes.
- **Reduction in Number of Collisions:** After all time steps, a sum reduction using *MPI\_reduction* is done to calculate total number of collisions.

## Tests

Twelve test cases were provided by the staff. Each of them was executed while varying the number of MPI tasks (1, 2, 4, 8, 16, 32) and the number of CPUs per task (1, 2, 4, 6, 8). This resulted in a wide range of configurations to evaluate the scalability of the implementation. All tests were submitted to the laboratory cluster using the Slurm workload manager. A typical job submission script looked like the following:

```
#!/usr/bin/env bash
#SBATCH --job-name=$job_name
#SBATCH --output=$output_file
#SBATCH --error=$output_file
#SBATCH --ntasks=$ntasks
#SBATCH --cpus-per-task=$cpus_per_task
#SBATCH --exclusive
#SBATCH --exclude=lab5p[1-20]
#SBATCH --exclude=lab0p[1-9]
#SBATCH --ntasks-per-node=1
srun parsim-mpi $params
```

Machines from the lab5 and lab0 groups were excluded using the `--exclude` option, as they did not have mpicc installed. The `--exclusive` flag was also used to ensure that each job would only run when the allocated node was in idle state.

## Results

### Execution Time Analysis

Different considerations might be retrieved from the execution time measured. Below follows insights of this results.

#### Execution Time vs Number of Tasks

Figure 1 shows the execution time trend as the number of MPI tasks increases, with each task using exactly one CPU associated to the test parameters 3 5000 50 1000000 500.

- 1 to 2 Tasks: A slight *increase* in execution time is observed, from approximately 78s to 84s. This is likely due to the problems discussed in , such as using different machines in each test.
- 2 to 4 Tasks: A speed up of 1.58x was obtained, which is below the ideal, probably due to the MPI communication overhead.
- 4 to 32 Tasks: Execution time continues to decrease, reaching a 5.52x speed up, showing near-linear scalability. However, the execution time continues to decrease with 32 tasks, suggesting that the implementation probably has a good scalability.

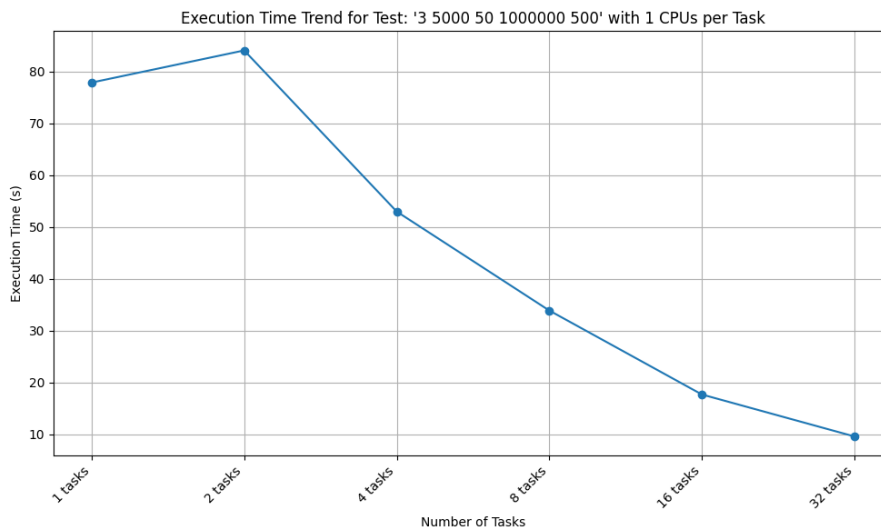


Figure 1: Execution Time Trend for Test 3 5000 50 1000000 500 with 1 CPU per Task

#### Speed Up vs Number of CPUs per Tasks

The speed up curve associated with the test parameters 1 5000 100 1000000 100 was analyzed across two configurations: one task and eight tasks.

In a single-task configuration, the execution time decreased as the number of CPUs increased, as expected, showing a super-linear speed up from 1 to 2 CPUs of 3.23x, probably due to better spatial/temporal locality in cache or by hardware heterogeneity between tests. However, an anomaly appears at 6 CPUs, where the execution time increases before dropping again at 8 CPUs, probably due to network latency and computer heterogeneity.

When scaling to eight tasks, the system achieved a super-linear speed up of 8.21x, likely due to better spatial/temporal cache locality or hardware heterogeneity across tests run. However, increasing the number of CPUs per task results in a suboptimal speed up. This is likely caused by serial and communication components in the code that cannot be parallelized with OpenMP, thus limiting overall performance gains (Karp-Flatt Metric).

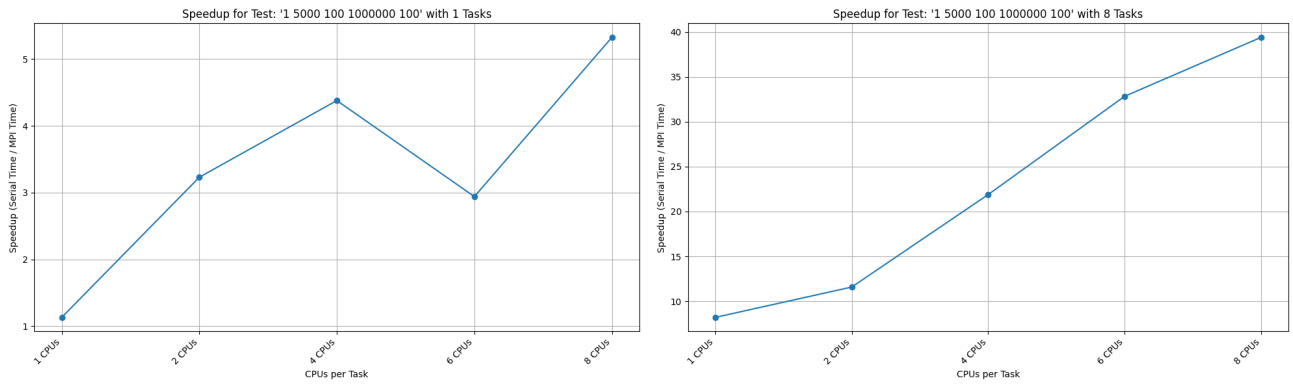


Figure 2: Speedup for Test 1 5000 100 1000000 100 with 1 task (left) and 8 task (right)

## Performance Considerations

Despite the optimizations and structured parallel design, the implementation does not achieve perfect speedup, the main reasons are:

- **Machines Heterogeneity:** Each run may involve different machines, and performance can vary significantly due to differences in hardware specification.
- **Network Latency:** Communication between machines relies on a basic TCP/IP network, which operates on a best-effort delivery model and introduces non-predictable latency.
- **Number of Physical Cores:** When increasing the number of CPUs per Task from 6 to 8, does not yield proportional speedup, as most machines in the laboratory environment have only 6 physical cores.
- **MPI functions and OMP directives:** Some MPI functions (e.g., `MPI_Recv`) and OpenMP synchronization points (e.g., barriers and dynamic scheduling) introduce overhead due to coordination among threads or processes.
- **Load Balancing between nodes:** Since the cells are static assigned for each node, if one node needs to performs much more computation than our neighbors, this can delays the progression of the algorithm.

## Conclusion

This project explores the parallel and distributed computing techniques through the implementation of a particle simulation based on the  $n$ -body problem.

The MPI implementation, structured around Foster's Design Methodology, demonstrated good scalability when running on multiple tasks and CPUs, particularly for larger problem sizes.

Linear scalability was not always achieved. This was primarily due to real-world limitations such as hardware heterogeneity, non-uniform network latency, and the MPI/OpenMP abstractions overhead. These factors are particularly evident when jobs span across different nodes, where inconsistent performance was observed.

The simulation scaled effectively up to 32 processors and exhibited substantial reductions in execution time. The use of checkerboard-style agglomeration, communication-computation overlap and buffer reuse contributed significantly to this outcome.

Future improvements could involve implementing dynamic load balancing strategies between nodes and adding redundancy computation between nodes to reduce the communication.